

Layouts and Partial with EJS

Section	9 — Views, Templates & Static Assets
Subtopic	09-03 — Layouts and Partial
Duration	~2 hours (part of the 6–8 hr section)
Tags	Theory Practical TypeScript
Topics covered	Running TypeScript with ts-node, EJS include syntax, partial files, shared layout pattern, passing locals to partials, dynamic content rendering, express-ejs-layouts

Learning Objectives — This Subtopic

- Set up and run a TypeScript + Express project using `ts-node` and `nodemon`
- Extract repeated HTML into partial files and include them using EJS `<%- include() %>`
- Construct a consistent page layout (shell) shared across all views
- Pass data into partials at the point of inclusion
- Render dynamic content within a layout using `res.locals` and per-render locals
- Evaluate the `express-ejs-layouts` package as a formal layout abstraction

Running TypeScript in an Express Project

All code examples in this subtopic are written in TypeScript. TypeORM (introduced in Section 10) is a TypeScript-first ORM, so we adopt TypeScript now to establish the project structure you will continue using. The runtime tool is `ts-node`, which compiles and executes TypeScript files directly without a separate build step — ideal during development.

Project Initialisation

Create a fresh project directory and initialise it:

```
mkdir views-demo && cd views-demo
```

```
npm init -y
```

```
npm install express ejs
```

```
npm install --save-dev typescript ts-node nodemon @types/node @types/express @types/ejs
```

TypeScript Configuration

Generate a `tsconfig.json` :

```
npx tsc --init
```

Replace the generated file with this focused configuration:

`tsconfig.json`

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "lib": ["ES2020"],
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "resolveJsonModule": true
  },
  "include": ["src/**/*.ts"],
  "exclude": ["node_modules", "dist"]
}
```

Key Insight: `"module": "commonjs"` is set here because Node.js and ts-node work with CommonJS by default. TypeScript will compile `import / export` syntax into `require() / module.exports` under the hood. This means you write modern TypeScript `import` syntax, but the runtime sees CommonJS — no mismatch.

NPM Scripts for Development

package.json (scripts section)

```
{
  "scripts": {
    "dev": "nodemon --watch src --ext ts --exec ts-node src/app.ts",
    "build": "tsc",
    "start": "node dist/app.js"
  }
}
```

What each script does:

`dev` — Nodemon watches the `src/` directory for `.ts` file changes. On each change, it re-runs `ts-node src/app.ts`, which compiles and starts the server in one step. No manual restarts needed.

`build` — Compiles the entire project to `dist/` for production.

`start` — Runs the compiled JavaScript from `dist/` in production.

Running the Development Server

```
npm run dev
```

Output:

```
[nodemon] 3.x.x
[nodemon] watching path(s): src/**/*.ts
[nodemon] watching extensions: ts
[nodemon] starting `ts-node src/app.ts`
Server running on http://localhost:3000
```

Every time you save a `.ts` file, nodemon restarts automatically. For production, run `npm run build` first, then `npm start`.

Important: Never run `ts-node` in production. It compiles on-the-fly and adds overhead. Always compile to JavaScript first with `tsc`, then execute with `node`.

The Problem: Duplicated HTML Across Views

A typical multi-page web application has a consistent visual shell — a navigation bar at the top, a footer at the bottom, and perhaps a sidebar. Without partials, every EJS view file duplicates that HTML verbatim. A change to the navigation means touching every single view file. This is a maintenance problem.

Analogy — Think of a shared layout like a PowerPoint slide master. You define the common elements once — logo, background, footer — and every slide inherits them automatically. Individual slides only define their unique content. Partials and layouts apply the same principle to HTML views.

EJS solves this with the `include` directive: a view file can pull in another EJS file at render time, merging them into a single HTML response. The included files are called **partials**.

EJS Include Syntax

EJS provides two tags for including other files:

Tag	Behaviour	When to use
<code><%- include('path') %></code>	Inserts the partial's content unescaped (raw HTML)	Always — for HTML partials
<code><%= include('path') %></code>	Inserts content escaped (turns HTML to plain text)	Never for partials — would render tags as visible text

Important: Always use `<%-` (dash, not equals) when including partials. Using `<%=` will HTML-escape the partial, causing your navigation markup to appear as raw text in the browser.

The path passed to `include()` is relative to the current view file's location, not to the project root.

Stage 1 — A Minimal Include

Start with the simplest possible case: one view including a single partial. First, the project structure at this stage:

```
views-demo/  
├─ src/  
│   └─ app.ts  
└─ views/  
    ├── index.ejs  
    └── partials/  
        └─ footer.ejs
```

views/partial/footer.ejs

```
<footer style="border-top: 1px solid #ccc; margin-top: 40px; padding: 12px 0; color: #666;">  
  <p>&copy; 2024 My App. All rights reserved.</p>  
</footer>
```

views/index.ejs

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8" />  
  <title>Home</title>  
</head>  
<body>  
  <h1>Welcome</h1>  
  <p>This is the home page.</p>
```

```
<%- include('partials/footer') %>
</body>
</html>
```

src/app.ts

```
import express, { Request, Response } from 'express';
import path from 'path';

const app = express();

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, '..', 'views'));

app.get('/', (req: Request, res: Response) => {
  res.render('index');
});

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

Output:

```
Welcome
This is the home page.
_____
© 2024 My App. All rights reserved.
```

EJS reads `index.ejs`, encounters the `include` call, reads `partials/footer.ejs` from the file system, splices its rendered content in at that position, and returns the merged HTML. The browser receives one complete HTML document — the inclusion happens entirely on the server.

Building a Shared Layout

The most practical pattern with plain EJS is the **header + footer wrap**: extract the opening HTML shell (doctype, head, opening body, navigation) into a header partial and the closing elements (footer, closing body/html) into a footer partial. Every view then sandwiches its unique content between those two includes.

Stage 2 — Header and Footer Partial

Expand the folder structure:

```
views-demo/  
├─ src/  
│   └─ app.ts  
└─ views/  
    ├── index.ejs  
    ├── about.ejs  
    └─ partials/  
        ├── header.ejs  
        └─ footer.ejs
```

views/partial/header.ejs

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8" />  
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />  
  <title><%= title %></title>  
  <style>  
    body { font-family: sans-serif; max-width: 900px; margin: 0 auto; padding: 0 20px; }  
    nav { background: #333; padding: 12px; margin-bottom: 24px; }  
    nav a { color: #fff; text-decoration: none; margin-right: 16px; }  
  </style>  
</head>  
<body>  
  <nav>
```

```
<a href="/">Home</a>
<a href="/about">About</a>
</nav>
```

Note: The header partial deliberately does *not* close `<body>` or `<html>`. Those tags are closed by the footer partial. The three files (header, content, footer) together form one valid HTML document.

views/partials/footer.ejs

```
<footer style="border-top: 1px solid #ddd; margin-top: 48px; padding: 16px 0; color: #666;
font-size: 0.9em;">
  <p>&copy; 2024 My App</p>
</footer>
</body>
</html>
```

views/index.ejs

```
<%- include('partials/header', { title: 'Home' }) %>

<h1>Welcome to My App</h1>
<p>This is the home page. Only this content changes between pages.</p>

<%- include('partials/footer') %>
```

views/about.ejs

```
<%- include('partials/header', { title: 'About' }) %>

<h1>About Us</h1>
<p>We build great things with Node.js and Express.</p>

<%- include('partials/footer') %>
```

Update `app.ts` to serve both routes:

src/app.ts

```
import express, { Request, Response } from 'express';
import path from 'path';

const app = express();

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, '..', 'views'));

app.get('/', (req: Request, res: Response) => {
  res.render('index');
});

app.get('/about', (req: Request, res: Response) => {
  res.render('about');
});

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

Output:

```
GET /      → renders index.ejs  (title: "Home",  nav + h1 "Welcome to My App" + footer)
GET /about → renders about.ejs   (title: "About", nav + h1 "About Us" + footer)
```

Both pages share an identical navigation bar and footer. To add a third navigation link later, you edit `partials/header.ejs` once — all pages update automatically.

Passing Data to Partials

The `include()` call accepts an optional second argument: a plain JavaScript object whose properties become local variables inside the included file. This is how `title` is made available inside `header.ejs` in the example above.

Data Scoping Rules

Understanding exactly which data is available inside a partial prevents subtle bugs:

Data source	How it reaches a partial	Example
<code>res.render('view', { key: val })</code>	Available in the parent view and all its includes automatically	<code>res.render('index', { user: 'Alice' })</code>
<code>res.locals.key = val</code>	Available in all views and partials for the lifetime of the request	Set in middleware: <code>res.locals.appName = 'My App'</code>
<code>include('partial', { key: val })</code>	Available only inside that partial — overrides or supplements parent scope	<code><%- include('header', { title: 'Home' }) %></code>

Stage 3 — Using `res.locals` for Application-Wide Data

Data like the application name, the logged-in user's name, or a flash message should be available in every view without passing it explicitly on every `res.render` call. Set it on `res.locals` in middleware instead.

src/app.ts

```
import express, { Request, Response, NextFunction } from 'express';
import path from 'path';

const app = express();

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, '..', 'views'));

// Middleware: attach app-wide locals before any route runs
app.use((req: Request, res: Response, next: NextFunction) => {
  res.locals.appName = 'My App';
  res.locals.currentYear = new Date().getFullYear();
  next();
});

app.get('/', (req: Request, res: Response) => {
  // 'user' is page-specific; 'appName' and 'currentYear' are already on res.locals
  res.render('index', { user: 'Alice' });
});

app.get('/about', (req: Request, res: Response) => {
```

```
res.render('about');
});

app.listen(3000, () => console.log('Server running on http://localhost:3000'));
```

Now update the partials to consume these values:

views/partials/header.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title><%= title %> | <%= appName %></title>
  <style>
    body { font-family: sans-serif; max-width: 900px; margin: 0 auto; padding: 0 20px; }
    nav { background: #333; padding: 12px; margin-bottom: 24px; }
    nav a { color: #fff; text-decoration: none; margin-right: 16px; }
  </style>
</head>
<body>
  <nav>
    <span style="color: #ccc; margin-right: 20px;"><%= appName %></span>
    <a href="/">Home</a>
    <a href="/about">About</a>
  </nav>
```

views/partials/footer.ejs

```
<footer style="border-top: 1px solid #ddd; margin-top: 48px; padding: 16px 0; color: #666;">
  <p>&copy; <%= currentYear %> <%= appName %></p>
</footer>
</body>
</html>
```

views/index.ejs

```
<%- include('partials/header', { title: 'Home' }) %>

<h1>Welcome, <%= user %>!</h1>
<p>This data came from res.render: <%= user %></p>
```

```
<p>This data came from res.locals: <%= appName %> / <%= currentYear %></p>
```

```
<%- include('partials/footer') %>
```

Output:

```
Browser tab title: "Home | My App"
Nav bar:           "My App Home About"
Page heading:      "Welcome, Alice!"
Footer:            "@ 2024 My App"
```

`title` is passed at the point of include (per-page), while `appName` and `currentYear` are available everywhere via `res.locals` without any extra passing. The partial reads both transparently.

Dynamic Content Rendering in Partials

Partials are full EJS templates — they can contain loops, conditionals, and any EJS expression. This makes them suitable for more than just static navigation bars.

Conditional Rendering: Active Navigation Link

A common requirement is highlighting the current page's link in the navigation. Pass the current page identifier from the route and use it inside the header partial.

src/app.ts (route update)

```
app.get('/', (req: Request, res: Response) => {
  res.render('index', { user: 'Alice', currentPage: 'home' });
});

app.get('/about', (req: Request, res: Response) => {
  res.render('about', { currentPage: 'about' });
});
```

views/partials/header.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title><%= title %> | <%= appName %></title>
  <style>
    body { font-family: sans-serif; max-width: 900px; margin: 0 auto; padding: 0 20px; }
    nav { background: #333; padding: 12px 20px; margin-bottom: 24px; }
    nav a { color: #aaa; text-decoration: none; margin-right: 16px; font-size: 0.95em; }
    nav a.active { color: #fff; font-weight: 600; border-bottom: 2px solid #4CAF50; padding-
bottom: 2px; }
  </style>
</head>
<body>
  <nav>
    <a href="/" class="<%= currentPage === 'home' ? 'active' : '' %>">Home</a>
    <a href="/about" class="<%= currentPage === 'about' ? 'active' : '' %>">About</a>
  </nav>
```

Output:

```
On GET /      → "Home" link gets class="active" (white + green underline)
On GET /about → "About" link gets class="active"
```

Looping Inside a Partial: Notification List

A notifications partial receives an array and renders each item — a common UI pattern for dashboards and admin interfaces.

views/partials/notifications.ejs

```
<% if (notifications && notifications.length > 0) { %>
  <div style="background: #fff3cd; border: 1px solid #ffc107; border-radius: 4px; padding: 12px
16px; margin-bottom: 20px;">
    <strong>Notifications (<%= notifications.length %>)</strong>
    <ul style="margin: 8px 0 0 0;">
      <% notifications.forEach(function(note) { %>
        <li><%= note %></li>
      <% }); %>
```

```
</ul>
</div>
<% } %>
```

views/index.ejs

```
<%- include('partials/header', { title: 'Home' }) %>

<%- include('partials/notifications', {
  notifications: ['Your profile is incomplete.', 'Welcome back, ' + user + '!']
}) %>

<h1>Welcome, <%= user %>!</h1>

<%- include('partials/footer') %>
```

Output:

```
Notifications (2)
• Your profile is incomplete.
• Welcome back, Alice!
```

Welcome, Alice!

When `notifications` is an empty array or undefined, the `if` block inside the partial evaluates to false and the box does not render — no empty container is left in the HTML.

Formal Layout Support: `express-ejs-layouts`

The header/footer include pattern works reliably, but it requires every view to explicitly call both includes. The `express-ejs-layouts` package inverts this model: you define one layout file that declares where page-specific content goes, and views no longer need to include anything. The middleware handles the wrapping automatically.

Installation

```
npm install express-ejs-layouts
```

```
npm install --save-dev @types/express-ejs-layouts
```

Note: Type definitions for `express-ejs-layouts` may not be available on DefinitelyTyped. If `@types/express-ejs-layouts` is not found, create a local declaration file to satisfy the TypeScript compiler — shown below.

```
src/types/express-ejs-layouts.d.ts
```

```
declare module 'express-ejs-layouts';
```

Stage 4 — Layout File and Views

Update the folder structure for this approach:

```
views-demo/
├─ src/
│   └─ app.ts
│   └─ types/
│       └─ express-ejs-layouts.d.ts
└─ views/
    └─ layout.ejs      <-- the single layout shell
    └─ index.ejs      <-- content only, no includes needed
    └─ about.ejs      <-- content only, no includes needed
```

```
views/layout.ejs
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title><%= title %> | <%= appName %></title>
  <style>
    body { font-family: sans-serif; max-width: 900px; margin: 0 auto; padding: 0 20px; }
    nav { background: #333; padding: 12px 20px; margin-bottom: 24px; }
```

```
    nav a { color: #aaa; text-decoration: none; margin-right: 16px; }
  </style>
</head>
<body>
  <nav>
    <a href="/">Home</a>
    <a href="/about">About</a>
  </nav>

  <!-- The view's content is injected here -->
  <%- body %>

  <footer style="border-top: 1px solid #ddd; margin-top: 48px; padding: 16px 0; color: #666;">
    <p>&copy; <%= currentYear %> <%= appName %></p>
  </footer>
</body>
</html>
```

`<%- body %>` is the single injection point. The middleware renders the requested view, captures its output, assigns it to `body`, then renders the layout with that value in place.

views/index.ejs

```
<h1>Welcome, <%= user %>!</h1>
<p>No layout includes needed – the shell is handled automatically.</p>
```

views/about.ejs

```
<h1>About Us</h1>
<p>We build things with Node.js and Express.</p>
```

src/app.ts

```
import express, { Request, Response, NextFunction } from 'express';
import path from 'path';
import ejsLayouts from 'express-ejs-layouts';

const app = express();

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, '..', 'views'));
```

```
// Register the layout middleware – must come after view engine setup
app.use(ejsLayouts);

// Tell the middleware which file is the layout
app.set('layout', 'layout');

// App-wide locals
app.use((req: Request, res: Response, next: NextFunction) => {
  res.locals.appName = 'My App';
  res.locals.currentYear = new Date().getFullYear();
  next();
});

app.get('/', (req: Request, res: Response) => {
  res.render('index', { title: 'Home', user: 'Alice' });
});

app.get('/about', (req: Request, res: Response) => {
  res.render('about', { title: 'About' });
});

app.listen(3000, () => console.log('Server running on http://localhost:3000'));
```

Output:

```
GET /      → layout.ejs wraps index.ejs content → full page with nav + "Welcome, Alice!" + footer
GET /about → layout.ejs wraps about.ejs content  → full page with nav + "About Us" + footer
```

Disabling the Layout for Specific Routes

Some responses — AJAX partial refreshes, print views, API error pages — should not be wrapped in the layout. Pass `layout: false` in the render locals to opt out:

src/app.ts (snippet)

```
app.get('/print', (req: Request, res: Response) => {
  // Render the view without any layout wrapper
  res.render('print-view', { layout: false, title: 'Print View' });
});
```

Output:

GET /print → only the content of print-view.ejs is returned; no nav, no footer

Choosing Between the Two Approaches

Criterion	Header/Footer Includes	express-ejs-layouts
Extra dependency	None — works with EJS only	Requires <code>express-ejs-layouts</code>
View file boilerplate	Each view must call two includes	Views contain only their unique content
Multiple layout support	Easy — just include different header files	Possible via <code>layout: 'other-layout'</code> in locals
Layout opt-out	Simply omit the includes	Pass <code>layout: false</code>
Transparency	Explicit — student can trace exactly what happens	Implicit — middleware wraps silently
Recommended for	Smaller projects, learning context, fine-grained control	Larger projects with many uniform views

Key Insight: Both approaches produce identical HTML output. The difference is organisational. For this module's practicals, the header/footer include pattern is sufficient and more transparent. `express-ejs-layouts` becomes valuable when a project has 10+ views that all share the same shell.

Summary

Concept	Key Point
---------	-----------

Running TypeScript	<code>ts-node</code> compiles and executes on the fly; <code>nodemon --exec ts-node</code> restarts on file changes. Never use <code>ts-node</code> in production.
EJS include syntax	Always <code><%- include('path') %></code> (dash, not equals) to insert raw HTML without escaping
Partial files	Isolated EJS fragments stored in <code>views/partials/</code> ; included in parent views at render time
Header/footer layout pattern	Split the HTML shell into header and footer partials; views sandwich unique content between two includes
Passing data to partials	Second argument to <code>include()</code> is a plain object; properties become local variables inside the partial
<code>res.locals</code>	Middleware-set properties available in all views and partials for the current request — no manual passing required
Dynamic rendering in partials	Partials support full EJS — conditionals, loops, expressions — not just static HTML
<code>express-ejs-layouts</code>	Inverts the model: layout wraps views automatically via <code><%- body %></code> ; opt out with <code>layout: false</code>

Interview Questions

1. What is the difference between `<%- include() %>` and `<%= include() %>` in EJS?

Expected answer: `<%-` renders the included content as raw, unescaped HTML — necessary for partials that contain markup. `<%=` HTML-escapes the output, so angle brackets become `<` / `>` and are displayed as visible text in the browser, not as rendered HTML.

2. Why should `ts-node` not be used in production?

Expected answer: `ts-node` compiles TypeScript at startup and on each file load, adding CPU and memory overhead that is unnecessary when the code does not change. Production deployments should compile with `tsc` once and run the resulting plain JavaScript with `node`.

3. You have a navigation bar that must appear on every page. A new developer on your team has copy-pasted the nav HTML into each of the 12 view files. What is the problem with this approach, and how would you fix it?

Expected answer: Any change to the navigation — a new link, a style update — requires editing all 12 files, creating high maintenance burden and risk of inconsistency. The fix is to extract the nav HTML into a partial (e.g., `views/partials/header.ejs`) and replace each copy with a single `<%- include('partials/header') %>` call.

4. What is `res.locals` and how does it differ from passing data directly to `res.render()`?

Expected answer: `res.locals` is a request-scoped object populated in middleware whose properties are automatically available to all views and partials for that request, without explicit passing. Data passed to `res.render('view', { key: val })` is specific to that render call and only available in that view (though it propagates to its includes). `res.locals` is appropriate for app-wide data like user session info or the application name.

5. How does `express-ejs-layouts` change the rendering pipeline compared to the manual header/footer include pattern?

Expected answer: With the manual pattern, each view is responsible for including its own header and footer. With `express-ejs-layouts`, the middleware intercepts `res.render()`, renders the view to a string, injects that string into `body`, then renders the layout template with it — the view itself does not need any include calls. The resulting HTML is the same; only the responsibility for wrapping changes.

6. Can you pass an array to a partial and loop over it? Give a use case.

Expected answer: Yes. The second argument to `include()` can contain any JavaScript value, including arrays. The partial uses standard EJS to iterate: `<% items.forEach(item => { %> ... <% }); %>`. A common use case is a notifications partial that receives an array of message strings and renders each as a list item.

7. Your header partial uses a variable `title` for the `<title>` tag. On one particular page, you forget to pass `title`. What happens at runtime?

Expected answer: EJS will throw a `ReferenceError: title is not defined` at render time, resulting in a 500 error. To guard against this, the partial can use a fallback: `<%= typeof title !== 'undefined' ? title : 'Default Title' %>`, or the route can always ensure `title` is passed via `res.locals`.

8. You have a route that returns a partial HTML fragment for an AJAX request — it should not include the full layout shell. How do you achieve this with `express-ejs-layouts`?

Expected answer: Pass `layout: false` in the locals object to `res.render()`: `res.render('fragment', { layout: false })`. The middleware detects this flag and skips the layout wrapping, returning only the rendered view content.

Practical Assignment — Multi-Page Site with Shared Layout

Objective: Build a three-page TypeScript + Express application that uses the header/footer partial pattern with active navigation highlighting, an application-wide local, and a dynamic partial.

Steps

1. Initialise a new project: `mkdir layout-practical && cd layout-practical && npm init -y`. Install `npm install express ejs` and `npm install --save-dev typescript ts-node nodemon @types/node @types/express`. Create a `tsconfig.json` as shown in the notes (`rootDir: src`, `outDir: dist`, `compilerOptions: { strict: true }`).
2. Add a `dev` script to `package.json`: `"dev": "nodemon --watch src --ext ts --exec ts-node src/app.ts"`.

3. Create the folder structure:

```
layout-practical/  
├─ src/  
│   └─ app.ts  
└─ views/  
    ├─ home.ejs  
    ├─ products.ejs  
    ├─ contact.ejs  
    └─ partials/  
        ├─ header.ejs  
        ├─ footer.ejs  
        └─ breadcrumb.ejs
```

4. In `src/app.ts`, set up Express with EJS, configure `res.locals.siteName = 'Shop Demo'` and `res.locals.currentYear` in middleware, then define three routes: `GET /` → renders `home` with `{ title: 'Home', currentPage: 'home' }`; `GET /products` → renders `products` with a `products` array of at least three strings and `currentPage: 'products'`; `GET /contact` → renders `contact` with `currentPage: 'contact'`.
5. Build `views/partials/header.ejs`: full HTML opening, a `<title>` using both `title` and `siteName`, a nav with three links (`/`, `/products`, `/contact`), and active class highlighting using `currentPage`.
6. Build `views/partials/footer.ejs`: a footer showing `currentYear` and `siteName`, then closing `</body></html>`.
7. Build `views/partials/breadcrumb.ejs`: accepts a `crumbs` array of objects with `label` and `href` properties, and renders them as: `Home > Products` with the last item not being a link.
8. In `views/products.ejs`, include the header, then include the breadcrumb partial passing `{ crumbs: [{ label: 'Home', href: '/' }, { label: 'Products', href: '' } ] }`, then loop over the `products` array and render each as an `<li>`, then include the footer.
9. Run `npm run dev` and verify all three pages render with the consistent shell, correct active nav link per page, and the product list on `/products`.

Expected Output

Output:

```
GET /      → title "Home | Shop Demo"
              nav: [Home*] Products Contact
              <h1>Welcome to Shop Demo</h1>
              footer: © 2024 Shop Demo

GET /products → title "Products | Shop Demo"
              nav: Home [Products*] Contact
              breadcrumb: Home › Products
              • Widget A
              • Widget B
              • Widget C
              footer: © 2024 Shop Demo

GET /contact → title "Contact | Shop Demo"
              nav: Home Products [Contact*]
              <h1>Contact Us</h1>
              footer: © 2024 Shop Demo

(* = active, highlighted link)
```

Bonus Challenge

Add a fourth route `GET /admin/dashboard` that renders a view *without* the standard navigation — the admin section has its own distinct sidebar layout. Implement this by creating `views/partials/admin-header.ejs` with a different navigation structure, and include it in `views/admin/dashboard.ejs` instead of the standard header. Confirm that visiting `/admin/dashboard` shows a completely different shell from the main site, while `/`, `/products`, and `/contact` are unaffected.

Notes on content additions/changes from syllabus: The syllabus for 09-03 lists "Shared layouts and includes" and "Dynamic content rendering" as bare bullet points. This document adds: (1) a dedicated TypeScript setup section covering `ts-node`, `nodemon`, and `tsconfig.json` — necessary because the syllabus had not explicitly established a TypeScript runtime by this point and future sections (TypeORM) require it; (2) the `res.locals` scoping table, which is implicit in the syllabus but critical for understanding partial data flow; (3) coverage of `express-ejs-layouts` as a formal layout mechanism, not mentioned by name in the syllabus but a direct implementation of "Shared layouts" that students frequently encounter in real projects.